

• [It Doesn't Taste Like APLs...](#)

[Applejak is a very, very small interpreter for a subset of the K programming language](#) which runs on the [Super CHIP-8](#) virtual machine; one of the most widely-ported VMs in existence. It was written in [Octo Assembly Language](#) as part of [Octojam 7](#).

To cram this program (and all of its working buffers) into 4kb of RAM, I had to cut many corners. Error-handling is rudimentary-to-nonexistent. There's no composition, projection, bracket-application, lambdas, or dictionary types. Data consists of numbers (unsigned bytes) and lists, which may contain at most 16 elements. The workspace has room for 64 "atoms"- numbers or lists- and intermediate values may be stored in one of 6 variables. Expect some bugs.

If you don't know anything about K or APL, please don't let this misshapen little stump of an interpreter color your opinions. Perhaps you could try one of [my other K interpreters](#)?

Controls

CHIP-8 does not have an ordinary keyboard interface; just a hex keypad. Thus, to enter an expression at Applejak's [REPL](#), you use a menu-driven tree-builder. Cycle between choices with the up and down cursor keys (or W/S), and confirm a selection by pressing the right cursor key (or D). On mobile devices, swipe up, down, or right, respectively. Choices that are not yet "confirmed" will be drawn with an inverted background.

At the top level, you can choose between the following types of tree nodes:

Symbol Meaning

- 

Number.
Add a single unsigned integer (0-255).
- 

Variable reference.
Choose one of the variables X,Y,Z,A,B,C and retrieve its value (if any).
- 

Variable assignment.
Choose a variable name, and enter an expression after the colon (:)
- 

Monadic verb.
Choose a monad (see below) followed by an input expression.
- 

Dyadic verb.
Choose a left expression (in parentheses), a dyad (see below), and a right expression.
- 

Adverb.
Choose a left expression (in parentheses), a dyad, an adverb (see below), and a right expression.
- 

List.
Enter a series of expressions, separated by semicolons (;) and terminated by a close parenthesis ()).

Monads

Symbol Meaning

- 

First.
Get the first element of a list. Identity for non-lists.
- 

Iota.
Generate a list from 0 up to a number N.
- 

Where.
For each number in a list, generate that many copies of its index.
For 1/0 lists, this produces a list of the indices of 1s.
- 

Reverse.
Generate a new list with the items of an input list in reverse-order.
- 

Not.
Logically invert any numeric arguments.
Recursively descends into list arguments. (Right-atomic.)
- 

Enlist.
Make a length-1 list containing any item.
- 

Length.
Get the number of items in a list.
- 

Random.
Given a number N, generate a list of N random bytes.

Dyads

The dyads which are noted as being "Atomic" will recursively penetrate to elements of lists. Adding a number to a list will add the number to each element of a list, while adding a list to a list will zip together matching elements of each list.

Symbol Meaning

	Plus. (Atomic.) Add two unsigned bytes.
	Minus. (Atomic.) Subtract two unsigned bytes.
	XOR. (Atomic.) Bitwise Exclusive OR of two unsigned bytes.
	Min. (Atomic.) Minimum of two unsigned bytes.
	Max. (Atomic.) Maximum of two unsigned bytes.
	Less. (Atomic.) 1 if X is less than Y; otherwise 0.
	More. (Atomic.) 1 if X is more than Y; otherwise 0.
	Equal. (Atomic.) 1 if X equals Y; otherwise 0.
	Join. Combine two lists or numbers into one flat list.
	Take. Generate a list of the first X items of list Y, repeating as necessary, or X copies of an atom Y.
	Drop. Generate a copy of Y after removing the first X items.
	At. Index elements Y from list X recursively. The shape of the result will always match the indexing structure Y.
	Match. 1 if X and Y are recursively identical; otherwise 0.
	Find. Return the index at a list X which matches Y; otherwise return an index 1 past the end of X.

Adverbs

Adverbs are verbs which modify the behavior of other verbs. In this case, they provide ways of applying dyads to every element of a list:

Symbol Meaning

	Over. Apply the verb between each element of a list, and return the final result.
	Scan. Like "Over", but accumulate a list of intermediate results.
	Each. Apply the verb between corresponding items of the left and right lists.

Errors

When Applejak crashes, it will attempt to display an error message. If you see an error message which does not look like one of the following, it suggests memory corruption. Whoopsie-doodle.

Symbol Meaning

	Workspace Full. Your calculation occupied too many atoms.
	Length Error. Tried to construct a list that is too large to fit in a workspace "slot".
	Type Error. Tried to apply a verb or adverb to inappropriate datatypes. For example, supplying a number to a verb which expects a list.